

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-211БВ-24

Студент: Отмахов Н.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 29.10.25

Москва, 2025

Постановка задачи

Вариант 5.

Отсортировать массив целых чисел при помощи четно-нечетной сортировки Бэтчера

Общий метод и алгоритм решения

Использованные системные вызовы:

- `int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg);` – Создает поток с заданными атрибутами, который начинает выполнение функции `start_routine`
- `int pthread_join(pthread_t thread, void **retval);` – ожидает завершения указанного потока.

Я реализовал программу, которая использует сортировку Бэтчера для сортировки массива целых чисел. На вход программа получает число, которое является максимальным количеством потоков, а также два имени файла. В первом находятся числа, а во второй производится вывод отсортированного массива.

Алгоритм сортировки итеративный. Он разбивает входной массив на равные участки, которые отдаются на сортировку разным потокам. Потом отсортированные участки сливаются. И на выходе получается отсортированный массив чисел.

Код программы

main.c

```
#include <fcntl.h>

#include <pthread.h>

#include <stdint.h>

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include "include/readfile.h"

#include "include/batcher_sort.h"

#include <time.h>

int main (int argc, char *argv[])

{

    if (argc != 4)

    {

        const char msg[] = "Using: ./program <num_of_threads> <input_file> <output_file>\n";

        write(STDOUT_FILENO, msg, sizeof(msg) - 1);
```

```

    exit(EXIT_SUCCESS);
}

const int32_t num_of_threads = atoi(argv[1]);
if (num_of_threads < 1)
{
    const char msg[] = "ERROR: Invalid num of threads\nNum of threads must be possitive integer\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

int32_t input_fd = open(argv[2], O_RDONLY);
if (input_fd == -1)
{
    const char msg[] = "ERROR: Invalid filename\nCan't open input file\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

int32_t output_fd = open(argv[3], O_WRONLY | O_CREAT);
if (output_fd == -1)
{
    const char msg[] = "ERROR: Invalid filename\nCan't open output file\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    close(input_fd);
    exit(EXIT_FAILURE);
}

uint32_t error_status;

char *file_data = read_from_file(input_fd, &error_status);
switch (error_status)
{
    case 1:
    {
        const char msg[] = "ERROR: Memory Error\nCan't get memory to read file\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
    }
}

```

```

        close(input_fd);

        close(output_fd);

        exit(EXIT_FAILURE);
    }

    break;

case 2:

{
    const char msg[] = "ERROR: Descriptor Error\nDescriptor can't read from input file\n";

    write(STDERR_FILENO, msg, sizeof(msg) - 1);

    close(input_fd);

    close(output_fd);

    exit(EXIT_FAILURE);
}

    break;
}

close(input_fd);


uint32_t nums_size = 0;

int32_t *nums = convert_to_int(file_data, &error_status, &nums_size);


switch (error_status)
{
    case 1:

    {
        const char msg[] = "ERROR: Memory Error\nCan't get memory to convert data\n";

        write(STDERR_FILENO, msg, sizeof(msg) - 1);

        close(output_fd);

        free(file_data);

        file_data = NULL;

        exit(EXIT_FAILURE);
    }

    break;

case 2:

{
    const char msg[] = "ERROR: Ptr Error\nFunction got wrong ptr\n";

```

```

        write(STDERR_FILENO, msg, sizeof(msg) - 1);

        close(output_fd);

        free(file_data);

        file_data = NULL;

        exit(EXIT_FAILURE);
    }

    break;

}

free(file_data);

file_data = NULL;


struct timespec start, end;

clock_gettime(CLOCK_MONOTONIC, &start);


batcher_sort(nums, nums_size, num_of_threads);


clock_gettime(CLOCK_MONOTONIC, &end);


double elapsed_ms = (end.tv_sec - start.tv_sec) * 1000.0
                    + (end.tv_nsec - start.tv_nsec) / 1e6;


char buf[100] = {};

snprintf(buf, 100, "Потоков: %u\tВремя: %.3f мс\n", num_of_threads, elapsed_ms);

write(STDOUT_FILENO, buf, 100);


write_nums_to_file(output_fd, nums, nums_size);

free(nums);

nums = NULL;


return 0;
}

```

batcher_sort.c

```
#include <stdint.h>
```

```
#include <unistd.h>
```

```
#include <pthread.h>
```

```
#include <stdlib.h>
```

```
typedef struct {
```

```
    int32_t *a;
```

```
    uint32_t start;
```

```
    uint32_t end;
```

```
    uint32_t j;
```

```
    uint32_t k;
```

```
} thread_task_t;
```

```
static inline void compare_and_swap(int32_t *a, uint32_t i, uint32_t j, int dir)
```

```
{
```

```
    if (dir == (a[i] > a[j]))
```

```
    {
```

```
        int32_t tmp = a[i];
```

```
        a[i] = a[j];
```

```
        a[j] = tmp;
```

```
    }
```

```
}
```

```
static void *worker_thread(void *arg)
```

```
{
```

```
    thread_task_t *task = (thread_task_t *)arg;
```

```
    uint32_t *a = (uint32_t *)task->a;
```

```
    uint32_t j = task->j;
```

```
    uint32_t k = task->k;
```

```
    for (uint32_t i = task->start; i < task->end; i++)
```

```
    {
```

```
        uint32_t ij = i ^ j;
```

```
        if (ij > i)
```

```
        {
```

```

        int dir = ((i & k) == 0) ? 1 : 0;

        compare_and_swap(task->a, i, ij, dir);
    }
}

return NULL;
}

static void apply_layer(uint32_t *a, uint32_t n, uint32_t k, uint32_t j, uint32_t max_threads)
{

    const uint32_t MIN_WORK_PER_THREAD = 131032;

    if (n < MIN_WORK_PER_THREAD * max_threads || max_threads <= 1)
    {

        for (uint32_t i = 0; i < n; i++)
        {
            uint32_t ij = i ^ j;

            if (ij > i)
            {
                int dir = ((i & k) == 0) ? 1 : 0;

                compare_and_swap(a, i, ij, dir);
            }
        }

        return;
    }

    uint32_t num_threads = (n + MIN_WORK_PER_THREAD - 1) / MIN_WORK_PER_THREAD;
    if (num_threads > max_threads) num_threads = max_threads;

    pthread_t *threads = calloc(num_threads, sizeof(pthread_t));
    thread_task_t *tasks = calloc(num_threads, sizeof(thread_task_t));

    if (!threads || !tasks)
    {

```

```

    free(threads);

    free(tasks);

    for (uint32_t i = 0; i < n; i++)
    {
        uint32_t ij = i ^ j;

        if (ij > i)
        {
            int dir = ((i & k) == 0) ? 1 : 0;

            compare_and_swap(a, i, ij, dir);
        }
    }

    return;
}

uint32_t chunk = (n + num_threads - 1) / num_threads;

int32_t ok = 1;

for (uint32_t t = 0; t < num_threads; t++)
{
    uint32_t start = t * chunk;

    uint32_t end = (t == num_threads - 1) ? n : start + chunk;

    if (start >= end) break;

    tasks[t] = (thread_task_t)
    {
        .a = a,

        .start = start,

        .end = end,

        .j = j,

        .k = k
    };

    if (pthread_create(&threads[t], NULL, worker_thread, &tasks[t]) != 0)
    {

```



```

        ok = 0;

        break;
    }
}

if (ok)
{
    for (uint32_t t = 0; t < num_threads; t++)
    {
        pthread_join(threads[t], NULL);
    }

    free(threads);

    free(tasks);

    return;
}

free(threads);
free(tasks);
for (uint32_t i = 0; i < n; i++)
{
    uint32_t ij = i ^ j;
    if (ij > i)
    {
        int dir = ((i & k) == 0) ? 1 : 0;

        compare_and_swap(a, i, ij, dir);
    }
}

}

void batcher_sort(int32_t *a, uint32_t n, uint32_t num_threads)
{
    if (n <= 1) return;

    if ((n & (n - 1)) != 0)
    {

```

```
    const char msg[] = "Error: n must be a power of two (1, 2, 4, 8, ...)\\n";  
    write(STDERR_FILENO, msg, sizeof(msg) - 1);  
    return;  
}  
  
if (num_threads == 0) num_threads = 1;  
  
for (uint32_t k = 2; k <= n; k <= 1)  
{  
    for (uint32_t j = k >> 1; j > 0; j >= 1)  
    {  
        apply_layer(a, n, k, j, num_threads);  
    }  
}  
}
```

Протокол работы программы

➤ ./program 1 test/numbers.txt test/output.txt

Потоков: 1 Время: 21573.949 мс

➤ ./program 2 test/numbers.txt test/output.txt

Потоков: 2 Время: 12016.940 мс

➤ ./program 4 test/numbers.txt test/output.txt

Потоков: 4 Время: 6133.317 мс

➤ ./program 8 test/numbers.txt test/output.txt

Потоков: 8 Время: 3692.070 мс

➤ ./program 16 test/numbers.txt test/output.txt

Потоков: 16 Время: 3271.295 мс

➤ ./program 32 test/numbers.txt test/output.txt

Потоков: 32 Время: 3457.700 мс

➤ ./program 64 test/numbers.txt test/output.txt

Потоков: 64 Время: 3326.228 мс

➤ ./program 1024 test/numbers.txt test/output.txt

Потоков: 1024 Время: 21582.657 мс

Вывод

В ходе данной лабораторной работы я научился работать с потоками. Разобрался с различными проблемами и нюансами при работе с ними (гонка). Также получил следующие выводы, которые я изображу в виде таблицы

Число потоков	Время исполнения (мс)	Ускорение	Эффективность
1	21573.949	1.00	1.00
2	12016.940	1.80	0.90
4	6133.317	3.52	0.88
8	3692.070	5.84	0.73
16	3271.295	6.59	0.41
32	3457.700	6.24	0.20
64	3326.228	6.49	0.10
1024	21582.657	0.99	0.001

Расчеты:

- Ускорение: $T1/Tn$, где $T1$ – время выполнения с 1 потоком,
 Tn – время выполнения с n потоками
- Эффективность: $Ускорение/n$

Анализ результатов:

По полученным данным видно, что ускорение растет, пока число потоков меньше или равно количеству логических процессоров. Когда потоков становится больше, наблюдается псевдопараллелизм. А если потоков значительно больше, чем ядер, то ускорение резко падает из-за накладных расходов на создание новых потоков.